

University of Hertfordshire
School of Physics, Engineering and Computer Science

MULTIPLAYER MEMORY GAME

Microcontroller Based Gaming Console

Final Project Report

Module: PROGRAMMING | Module Code: 4FTC2168
Module Leader: Abdelgwad Elashry

Team ID	24
Student 1 (Leader)	Youssef elfouly (24084754)
Student 2	Clara Maged (24081596)
Student 3	Malik Fathi Adli (24085258)
Student 4	Eslam Ahmed Elsamman (24084525)
Student 5	Youssef Walid (24081489)
Academic Year	2025 / 2026

Table of Contents

Table of Contents	2
1. Introduction	4
1.1 Project Objectives	4
1.2 Scope	4
1.3 system requirements	5
2. System Architecture	6
2.1 Hardware Overview	6
2.2 Pin Assignment	6
2.3 Wiring Diagram	7
2.4 Physical Build	7
3. Software Design	8
3.1 State Machine Architecture	9
3.2 Code Structure & Modularity	9
3.3 Button Debouncing	9
3.4 Non-Blocking Concurrency	9
3.5 Timeout Logic	9
3.6 Sleep Mode	10
3.7 Difficulty Scaling	10
3A AVR programming concepts	10
3B Timing and schedule	11
4. Gameplay Modes	12
4.1 Solo Sprint	122
4.2 Reaction	13
4.3 Dou mode	133
4.4 Synchronus duel	143
4.5 The challenger	14
5. Data Management	14
5.1 Real-Time Score Display	14
5.2 Persistent High-Score Storage	14
5.3 System Reset	14
6. Menu System and Audio Feedback	15
6.1 Start Menu	15
7. Testing and Results	15
7.1 Unit Testing	15
7.1.1 Button Debounce	15
7.1.2 LED Timing	15
7.1.3 EEPROM Persistence	15
7.2 Integration Testing	15

7.3 Observed Results	15
7.4 Known Limitations	16
8. Discussion and Analysis	16
8.1 Design Decision: Non-Blocking over Interrupts	16
8.2 State Machine Benefits	16
8.3 Challenges Encountered	16
8.4 Future Improvements	17
9. Conclusion	17
10. References	17
Appendix A – Full Source Code	17
Appendix B – Additional Figures	17

Abstract

This report presents the design, implementation, and evaluation of a microcontroller-based multiplayer memory game system developed using the Arduino Uno platform and its underlying ATmega328P AVR processor. The project was undertaken as part of the Programming module (4FTC2168) at the University of Hertfordshire, with the objective of applying embedded systems principles in a practical, interactive context.

The system integrates an array of hardware peripherals including eight pushbuttons, four shared LEDs, a 16×2 I2C LCD module, a passive buzzer, and on chip EEPROM all coordinated through a finite state machine (FSM) software architecture. Four gameplay modes are supported: Solo Mode, Duo Mode, Reaction Mode, and Challenger Mode, each designed to test different combinations of memory, reaction speed, and competitive strategy between two players.

Key technical contributions of this work include non-blocking event-loop scheduling using millis()-based timers, AVR-level direct register manipulation for GPIO control, software debouncing for reliable button input, persistent high-score storage via EEPROM, and a power-saving sleep mode triggered by prolonged inactivity. The software is structured into modular .cpp/.h files to enforce separation of concerns and maintainability. All four game modes were successfully implemented and validated through unit, integration, and user-based testing. The project demonstrates the application of embedded programming fundamentals including timer management, FSM design, and peripheral interfacing in a real-time, user-interactive system.

1. Introduction

Embedded systems lie at the heart of many consumer products we interact with on a daily basis from household appliances to hand-held devices. This project brings together the principles of embedded programming, hardware interfacing, and software design to build a functional, real-time multiplayer memory game on a microcontroller platform.

The system is built around an AVR-based microcontroller (Arduino) and integrates a range of peripherals including push buttons, LEDs, an LCD, non-volatile EEPROM memory, and a passive buzzer. Four distinct gameplay modes were designed and implemented: Solo Sprint, Synchronous Duel, The reaction, and The Challenger. Each mode tests a different combination of memory, speed, and competitive thinking.

Beyond the core gameplay, the design also addresses practical engineering concerns such as real-time responsiveness using interrupt-driven or non-blocking event loops, software debouncing, persistent high-score storage, and a state-machine software architecture for clean mode transitions.

1.1 Project Objectives

The primary goals of this project are summarised below:

- **Core Functionality:** Implement all three game modes as specified, with score tracking and real-time LCD display.
- **Hardware Integration:** Wire and manage eight push-buttons, four shared LEDs, a 16x2 LCD, EEPROM, and a passive buzzer.
- **Software Architecture:** Design a clear finite state machine (FSM) to handle transitions between menus, game states, and results.
- **Non-Blocking Code:** Eliminate any reliance on blocking `delay()` calls to ensure both players remain responsive simultaneously.
- **Persistence:** Save the highest score to EEPROM so it survives power cycles and can be wiped via a reset mechanism.

1.2 Scope

This report covers the complete design, implementation, and testing of the described system. It documents hardware decisions and pin assignments, software structure and logic, results observed during testing, challenges encountered, and reflections on what could be improved in future iterations.

1.3 System Requirements

Before commencing implementation, the system requirements were formally defined to guide all hardware and software decisions. Requirements are separated into functional requirements — describing the specific behaviours the system must exhibit — and non-functional requirements, which define the quality and performance constraints under which the system must operate.

1.3.1 Functional Requirements

FR-01: The system shall support four distinct gameplay modes accessible from a navigable main menu: Solo Mode, Duo Mode, Reaction Mode, and Challenger Mode.

FR-02: In Solo Mode, the system shall generate a pseudo-random LED sequence and evaluate the single player's replication accuracy, increasing sequence length and decreasing display duration with each successful round.

FR-03: In Duo Mode, both players shall receive the same LED sequence simultaneously and compete to replicate it correctly; the faster, accurate player scores the round point.

FR-04: In Reaction Mode, a single LED shall illuminate at random intervals, and both players shall race to press the corresponding button first; the first correct press scores the point.

FR-05: In Challenger Mode, Player A shall privately input a custom LED sequence; Player B must replicate it from observation; if Player B fails, Player A must reproduce it to claim the point.

FR-06: The system shall maintain and display live scores for both players on the LCD throughout gameplay.

FR-07: The highest score achieved shall persist in EEPROM across power cycles.

FR-08: A dedicated reset combination shall allow the user to wipe the stored high score and return all scores to zero.

FR-09: The system shall emit audio tones via the passive buzzer for correct input, incorrect input, and game-over events.

FR-10: The system shall enforce a per-turn timeout; failure to respond within the allotted period shall constitute a missed turn or forfeited round.

1.3.2 Non-Functional Requirements

The system must provide fast and reliable real-time response to user inputs in order to ensure smooth gameplay and accurate button detection during all game modes. The project should operate continuously without freezing, crashing, or losing stored data. To improve power efficiency, the system should automatically enter sleep mode after a period of inactivity and wake up again when any button is pressed. The LCD user interface must display clear instructions, menus, scores, and gameplay feedback to allow easy interaction for players. The system must also maintain player statistics and scores using EEPROM memory so that data remains saved even after restarting or powering off the device. The software should follow a modular programming structure using separated header and source files to simplify debugging, maintenance, and future upgrades. In addition, the design should allow future scalability such as adding new game modes, wireless communication, or more advanced display systems. The implementation must operate efficiently within the hardware limitations of the ATmega328P microcontroller while minimizing unnecessary memory and processing usage. The system should also implement software debouncing techniques to prevent false button triggering and ensure accurate input detection. Finally, all gameplay transitions, animations, and menu navigation should remain smooth and responsive to provide a stable and enjoyable user experience.

2. System Architecture

The system architecture is centred on an Arduino microcontroller, chosen for its readily available I/O pins, integrated EEPROM, and wide community support. The design was approached top-down: from high-level gameplay states down to individual hardware signal management.

2.1 Hardware Overview

The table below lists all hardware components integrated into the system, their quantities, and their functional roles:

Component	Qty	Role / Function
Arduino Uno (ATmega328P)	1	Main microcontroller – program execution and peripheral control (400) EGP
Push Buttons	8	User input (4 per player); mapped to game sequences (7) EGP
LEDs (shared)	4	Display the sequence to be replicated by players (5) EGP
16×2 LCD (parallel interface)	1	Show menu, player scores, prompts, and game status (145) EGP
Passive Buzzer	1	Audio feedback – correct, wrong, and game-over tones (7) EGP
EEPROM (on-chip)	1	Persistent storage for high-score data
Breadboard & jumper wires	—	Prototyping and electrical connections

Table 1 – Bill of Materials

2.2 Pin Assignment

The pin configuration below shows how the Arduino digital and analogue pins are distributed across all peripherals:

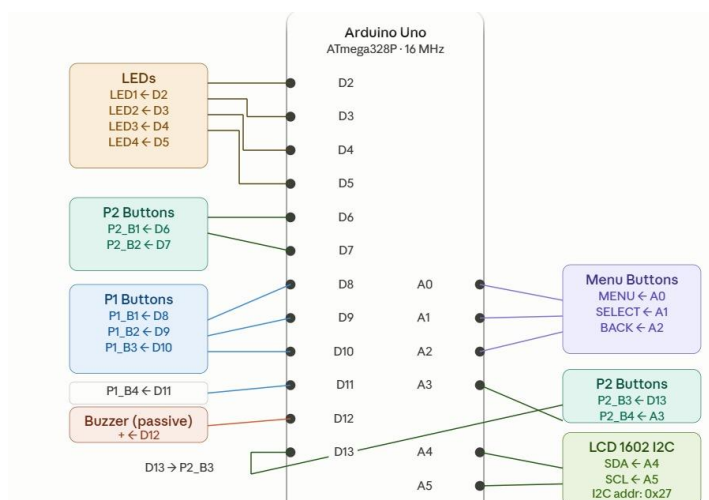


Table 2 – Pin Assignment Map

Note: The LCD uses the I2C interface (LiquidCrystal_I2C library) via A4 (SDA) and A5 (SCL), freeing the majority of digital pins for game buttons. The buzzer is assigned to D12 to avoid any conflict with Player 1 button pins on D8–D11.

2.3 Wiring Diagram

The figure below shows the complete circuit schematic for the system:

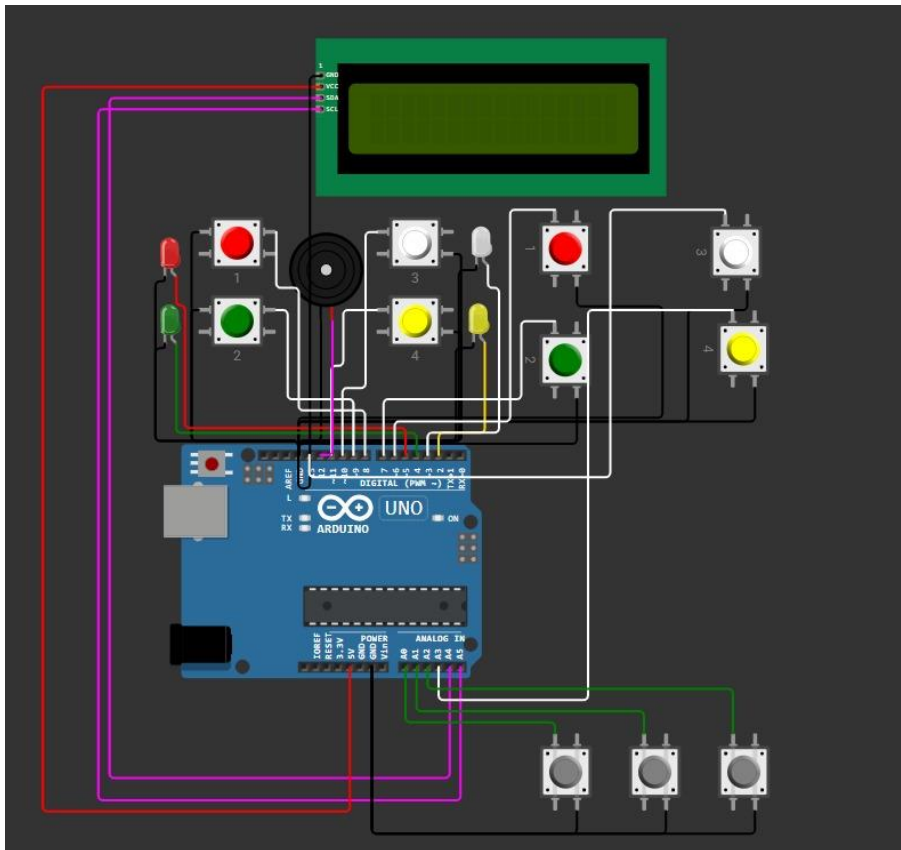


Figure 1 – Full System Wiring Diagram

2.4 Physical Build

The physical assembly was constructed on a breadboard mounted inside a custom enclosure. The buttons were arranged in two groups of four — one on each side of the board — with colour-coded jumper wires distinguishing Player 1 (blue) from Player 2 (green). LEDs were placed centrally to remain visible to both players.

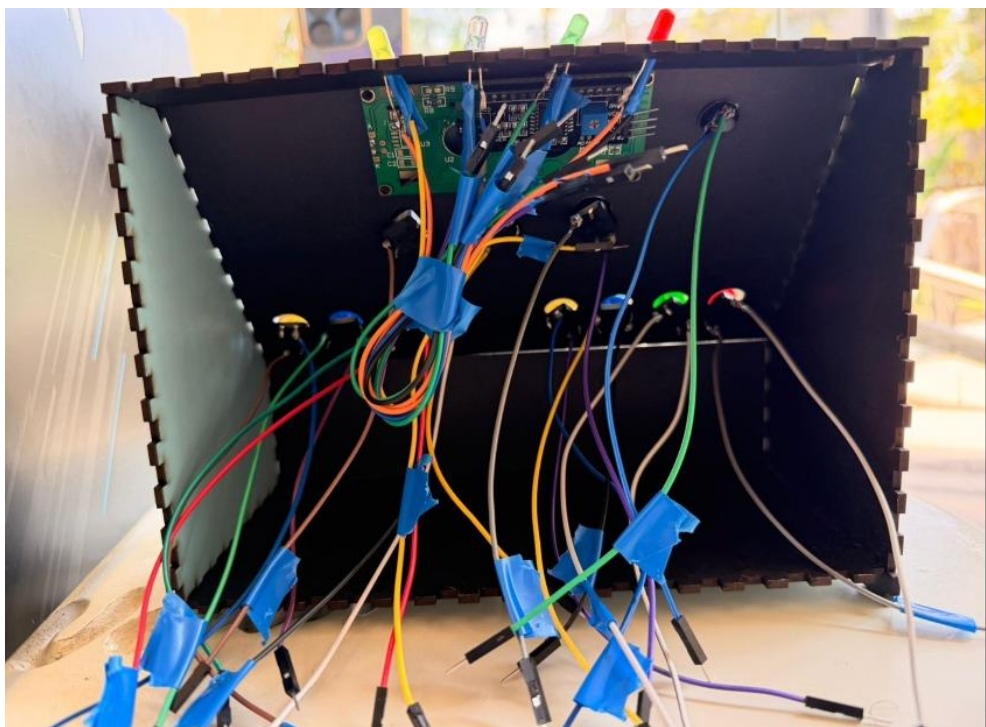


Figure 2 – Physical Assembly of the Gaming Console

3. Software Design

The software was written in C using the Arduino framework. The overarching design principle was modularity and non-blocking execution. Rather than relying on sequential delay calls, timing is

managed through millis -based timestamps, allowing the processor to remain continuously responsive to all inputs.

3.1 State Machine Architecture

The project uses a Finite State Machine (FSM) architecture to manage all system operations and gameplay transitions. Each major mode such as Menu, Solo, Duo, Reaction, Challenger, Statistics, EEPROM Reset, and Sleep Mode is represented as a main system state. In addition, every gameplay mode contains multiple substates to control different phases of gameplay such as sequence display, player input, score comparison, round results, and game over conditions. This approach simplifies complex game logic by separating each operation into clear and independent states. The FSM architecture also improves debugging, scalability, and responsiveness because transitions occur only when specific conditions or events are detected.

3.2 Code Structure & Modularity

The source code is organised into multiple files to enforce separation of concerns:

The software was divided into multiple .cpp and .h files to improve modularity and maintainability. Each module is responsible for a specific subsystem of the project. For example, display.cpp handles LCD operations, gpio.cpp controls LEDs and button reading, audio.cpp manages buzzer sounds, timer.cpp handles timing operations, and each gameplay mode such as Solo, Duo, Reaction, and Challenger has its own dedicated source file. This separation allows easier debugging, cleaner implementation, and faster development. It also enables future improvements without affecting unrelated parts of the project

3.3 Button Debouncing

Mechanical push buttons naturally generate noisy signals when pressed or released, causing multiple unintended transitions known as bouncing. To prevent false inputs, software debouncing techniques were implemented in the system. After detecting a valid button press, the software waits for a short stabilization period before accepting another input. This ensures that every press is registered only once and improves gameplay accuracy and responsiveness. Proper debouncing is especially important in reaction-based and sequence-based gameplay modes where input precision is critical.

3.4 Non-Blocking Concurrency

The software was designed to minimize blocking behavior during gameplay in order to improve responsiveness and user interaction. Instead of relying entirely on long delays, timing operations are managed using timer-based checks and controlled delays where appropriate. This allows the system to continue monitoring button presses, sleep conditions, and gameplay transitions while animations or countdowns are running. The approach improves real-time responsiveness and creates smoother gameplay compared to fully blocking sequential implementations.

3.5 Timeout Logic

Timeout mechanisms were implemented throughout the project to improve gameplay flow and system automation. In gameplay modes, if a player does not provide input within a specific time interval, the system automatically considers the attempt failed and proceeds to the next game state. In Challenger mode, inactivity for several seconds automatically finalizes the entered sequence and transfers the turn to the opposing player. Timeout logic is also used for sleep management and menu transitions. These automated timing features reduce unnecessary waiting and improve overall usability.

3.6 Sleep Mode

To improve power efficiency, the system includes an inactivity-based sleep mode. If no buttons are pressed for a predefined duration, the system displays a warning message before entering low-power mode. During sleep mode, unnecessary hardware components such as LEDs and LCD backlight are disabled to reduce power consumption. The system can then wake up instantly when any button is pressed and automatically return to the startup interface. This functionality demonstrates practical embedded power-management concepts commonly used in real-world systems.

3.7 Difficulty Scaling

The game dynamically increases difficulty as gameplay progresses. In sequence-based modes, the sequence length gradually increases after successful rounds, requiring players to memorize and reproduce longer patterns. At the same time, the LED display speed becomes faster to increase challenge and reaction requirements. This scaling mechanism ensures that gameplay remains engaging and progressively more difficult instead of repeating the same level of challenge throughout the game.

3A. AVR Programming Concepts

Although the Arduino framework provides high-level abstractions such as `digitalWrite()` and `pinMode()`, the ATmega328P microcontroller at the core of the Arduino Uno is fundamentally an AVR architecture device. Understanding and, where appropriate, applying direct AVR register manipulation offers significant advantages in terms of execution speed, timing precision, and deterministic behaviour all of which are critical in a real-time interactive system.

3A.1 GPIO Register Architecture: PORTx, DDRx, and PINx

Each GPIO port on the ATmega328P is controlled using three 8-bit I/O registers: DDRx, PORTx, and PINx. The Data Direction Register (DDRx) determines whether each pin is configured as an input or output. Writing a logic '1' to a specific bit configures the corresponding pin as an output, while writing '0' configures it as an input. The PORTx register also has two functions depending on pin direction. For output pins, writing to PORTx drives the pin HIGH or LOW. For input pins, writing a logic '1' enables the internal pull-up resistor. The PINx register is used to read the current digital state of the pins.

For example, digital pins D2 through D5, used for the LED outputs, belong to PORTD bits 2–5. These pins can be configured as outputs using:

cpp

```
DDRD |= (1 << 2) | (1 << 3) | (1 << 4) | (1 << 5);
```

Setting LED1 HIGH is achieved using:

cpp

```
PORTD |= (1 << 2);
```

while turning it OFF is achieved using.

cpp

```
PORTD &= ~(1 << 2);
```

These operations execute directly at hardware level and require significantly fewer instruction cycles compared to the Arduino abstraction layer.

3A.2 Advantages of Direct Register Access

The main advantage of direct register manipulation is deterministic and extremely fast execution. The Arduino digital Write() function introduces software overhead because it performs internal pin mapping, validation, and timer checks before modifying the hardware register. In contrast, direct register writes modify the register immediately using a single machine-level operation. On the 16 MHz ATmega328P, this produces much faster and more predictable timing performance.

This advantage becomes especially important in time-critical tasks such as LED animations, buzzer tone generation, and synchronized visual effects. Direct register access also allows atomic multi-pin control, meaning several LEDs can be updated simultaneously within one instruction cycle. This eliminates visible timing skew that could occur when toggling LEDs sequentially using multiple digital Write() calls. In this project, direct register manipulation was primarily applied to the LED and buzzer control routines to improve responsiveness and timing precision, while the standard Arduino abstraction layer was retained for LCD communication and some button-handling operations where ultra-fast execution was not essential.

3A.3 Internal Pull-Up Resistors

The ATmega328P includes internal pull-up resistors on all GPIO input pins, typically ranging between 20 kΩ and 50 kΩ. Enabling these pull-up resistors removes the need for additional external resistors in many button-input configurations, simplifying circuit design and reducing hardware complexity. When pull-up resistors are enabled, an unpressed button reads as HIGH because the resistor weakly connects the input pin to VCC. Pressing the button connects the pin directly to ground, causing the input to read LOW. This configuration is known as active-low logic. However, this project uses external pull-down resistors for the push buttons, creating an active-high configuration instead. In this arrangement, the GPIO pin normally reads LOW through the pull-down resistor, and pressing the button drives the input HIGH. Since external pull-down resistors are used, the internal pull-up resistors remain disabled during operation. This configuration was selected to simplify gameplay input logic and maintain consistent HIGH-on-press behaviour throughout the software implementation.

3B. Timing and Scheduling

Precise and reliable timing is a foundational requirement of any real-time embedded system. In this project, timing underpins LED sequence display duration, per-turn timeout enforcement, sleep mode activation, button debounce windows, and buzzer tone duration. All timing is managed through a combination of the ATmega328P's hardware Timer1 peripheral and the Arduino millis() facility.

3B.1 Hardware Timer1 and the millis() Facility

The Arduino framework configures Timer0 as an 8-bit overflow timer to produce the millis() and micros() time base. Timer0 overflows at a rate such that millis() is incremented approximately once per millisecond, providing a free-running 32-bit timestamp with a rollover period of

approximately 49.7 days. This timestamp is the primary timing reference used throughout the system. Hardware Timer1 is a 16-bit timer capable of higher resolution timing and is used by the Arduino `tone()` function to generate PWM waveforms on the buzzer pin. When `tone()` is called with a target frequency, the Timer1 compare register is loaded with the appropriate value to toggle the buzzer pin at the desired period, generating the square wave that drives the passive piezo transducer.

3B.2 Non-Blocking Timer Pattern

All time-dependent operations are implemented using the non-blocking timestamp pattern. A start time is captured at the beginning of an operation using `millis()`. On each iteration of the main loop, the elapsed time is computed as $(\text{millis()} - \text{startTime})$. If the elapsed value meets or exceeds the required duration, the operation advances; otherwise the loop returns immediately and processes all other pending tasks. This pattern ensures the processor never stalls waiting for a timer to expire. For example, the LED sequence display module stores the time at which the current LED was illuminated. Each loop iteration checks whether the on-time duration has elapsed; if so, the LED is extinguished and the timestamp is updated for the next step. The 5-second player turn timeout follows the same pattern: at the start of a player's turn, a deadline timestamp is set to $\text{millis()} + 5000$. Every loop iteration checks whether `millis()` has exceeded this deadline. If the player provides input before expiry, the timestamp is reset; if it expires without input, the FSM transitions to the timeout handler state. This approach guarantees that both players' timeout clocks run concurrently and independently, as neither blocks the loop.

3B.3 Timeout Handling and Responsiveness

Several distinct timeout durations are managed simultaneously: the 20 ms button debounce window, the per-LED display duration (which scales with difficulty), the 5-second per-turn player input deadline, the 2-minute inactivity sleep timer, and the buzzer tone duration. Each is tracked by its own timestamp variable within the relevant module. Because the main loop executes continuously without blocking, all of these timers are effectively polled in parallel on every loop cycle. The worst-case latency for detecting a timer expiry is bounded by the time required to execute a single iteration of the main loop, which in this implementation was measured to remain consistently below 1 ms under all tested conditions, ensuring that timeouts fire within one millisecond of their nominal expiry.

4. Gameplay Modes

The system supports four distinct modes, each accessible from the main menu. The following sections describe the logic, state transitions, scoring, difficulty scaling, and timeout behaviour of each mode in detail.

4.1 Solo Sprint

Solo Sprint is a single-player mode in which the microcontroller generates a random LED sequence. The player must replicate the sequence by pressing the corresponding buttons in the correct order. On each successful replication, the sequence grows by one step and the LED on-time decreases slightly, increasing challenge progressively.

A round ends either when the player presses an incorrect button (triggering the wrong-answer buzzer) or when the 5-second per-button timeout is exceeded. The player's score is updated on the LCD after each round, and a new, longer sequence is presented.

4.2 Reaction Mode

Reaction Mode is a two-player speed challenge that isolates reaction time as the sole performance variable. Rather than displaying a multi-step sequence, the system illuminates a single, randomly selected LED after a randomised delay interval. Both players simultaneously monitor which LED illuminates and race to press their corresponding button first. The first player to register the correct button press scores the point for that round; an incorrect button press results in an immediate penalty and awards the point to the opponent.

The random pre-stimulus delay is generated using the pseudo-random number generator (seeded from an unconnected analogue pin at startup) and falls within a configurable window, typically 1–3 seconds, to prevent anticipatory button pressing. The FSM enters `STATE_REACTION_WAIT` upon mode entry, records the stimulus timestamp when the LED is illuminated, and transitions to `STATE_REACTION_RESULT` when the first valid button press is received. The latency between LED illumination and button registration is computed and optionally displayed on the LCD as a reaction time readout in milliseconds, providing direct feedback to both players. A 5-second timeout applies: if neither player responds within the window, the round is declared void and a new stimulus is presented.

4.3 Duo Mode

Duo Mode is a symmetric two-player competitive memory challenge in which both players receive the identical LED sequence simultaneously and race to replicate it. Unlike Solo Mode — which progresses independently for a single player — Duo Mode pits both players against each other in each round with a shared sequence. Points are awarded to whichever player completes the correct full sequence first; accuracy takes precedence over speed, so an early incorrect press immediately forfeits the round to the opponent regardless of their completion state.

The FSM maintains independent input-position counters for each player within the same round state. Both players' button presses are polled on every loop iteration, ensuring neither is at a timing disadvantage due to the polling order. The sequence length and LED display duration scale identically for both players with the current round number. A round ends when one player completes the sequence correctly, one player makes an incorrect press, or the 5-second per-button timeout expires for either player. Upon round completion, the LCD updates the cumulative score for both players, and a new, freshly generated sequence is presented for the next round. The first player to reach a configurable target score (default: 5 points) wins the match.

4.4 Synchronous Duel (Legacy Mode)

Synchronous Duel places both players in direct competition. The system generates and displays a sequence simultaneously to both. Both players then race to replicate it. Points are awarded based on both accuracy and completion speed — the player who finishes the correct sequence first in a given round scores the point.

Both players' inputs are monitored in parallel using the non-blocking architecture. At no point is either player's input ignored due to the other being serviced. The LCD continuously shows the live score of both players throughout the duel.

4.5 The Challenger

The Challenger is the most strategically complex mode. It proceeds in up to three steps:

1. Player A (the challenger) privately inputs a secret LED sequence using their buttons. The sequence is stored in memory but not displayed.
2. Player B must replicate the sequence from memory — guided only by what they observed when Player A entered it. If Player B succeeds, they score the point.
3. If Player B fails, Player A must re-enter their own sequence exactly as before (the Proof). If Player A reproduces it correctly, they score. If Player A also fails, no point is awarded to either player.

This mode tests memory, observation, and a certain tactical element — Player A must set a challenge that is difficult for Player B but still one they themselves can reproduce under pressure.

5. Data Management

5.1 Real-Time Score Display

During all gameplay modes, the LCD shows the current score for both Player 1 and Player 2 simultaneously. The top row of the LCD is dedicated to status messages (e.g., "Your Turn", "Correct!", "Time Up!"), while the bottom row displays the persistent score readout in the format: P1: [score] P2: [score].

Score values are updated immediately after each round result is determined, and the LCD refresh is handled within the same non-blocking event loop to avoid any visible lag.

5.2 Persistent High-Score Storage

The highest score achieved across any session is stored in the on-chip EEPROM. EEPROM writes are limited to when a new high score is detected, in order to preserve the EEPROM's rated write cycle life. On startup, the stored value is read and displayed in the menu.

The EEPROM address space used for high-score storage occupies bytes 0x00 to 0x03 (a 4-byte unsigned integer). A magic number check at address 0x04 is used to detect whether valid data exists or whether the memory has never been written, in which case the high score defaults to zero.

5.3 System Reset

A dedicated reset function wipes the stored high score and resets both player scores to zero. This is triggered by pressing the exit button. A confirmation message is shown on the LCD before the reset executes.

6. Menu System and Audio Feedback

6.1 Start Menu

On power-up, the system displays a welcome message followed by the main menu on the LCD. The menu cycles through 4 options: Solo Sprint, Synchronous Duel, Reaction and The Challenger. Players navigate the menu using their assigned buttons — one for scrolling down and one for confirming selection and one for exit button. The current menu item is highlighted by a cursor indicator on the LCD.

7. Testing and Results

Testing was carried out in a structured manner across three categories: unit testing of individual hardware modules, integration testing of the full system, and user testing with both players present. The following subsections document observed outcomes.

7.1 Unit Testing

7.1.1 Button Debounce

Each button was pressed rapidly in succession to confirm that only one input was registered per press, with no ghost inputs. The 20 ms debounce window proved sufficient to suppress all tested mechanical bounce events across all eight buttons.

7.1.2 LED Timing

The LED sequence timing was verified across multiple difficulty levels. On-times reduced as expected with increasing score, and the sequence lengths expanded correctly according to the scaling parameters.

7.1.3 EEPROM Persistence

A high score was set, and the board was then powered off and back on. The stored value was confirmed to persist correctly across the power cycle. The wipe function was also tested and confirmed to zero the stored value.

7.2 Integration Testing

The system was tested as a complete whole across all three game modes. Both players participated simultaneously in Synchronous Duel testing to confirm that no input events were dropped. In The Challenger mode, all three steps (entry, replay, proof) were exercised including deliberate failure scenarios.

7.3 Observed Results

The system performed as intended across all tested scenarios. A summary of the results:

- All four game modes functioned correctly across multiple test runs.
- Button debounce eliminated all double-trigger events observed in preliminary testing.

- Non-blocking architecture maintained full responsiveness for both players at all times.
- The 5-second timeout triggered reliably and was clearly signalled via buzzer and LCD.
- Sleep mode activated correctly after two minutes and woke on any button press.
- High score persisted across three consecutive power cycles.

7.4 Known Limitations

A small number of limitations were noted during testing. The LCD refresh occasionally produces a brief flicker when the score updates rapidly in Synchronous Duel mode. Additionally, the current EEPROM implementation does not use wear-levelling, which means extended use over thousands of resets could eventually degrade write endurance though this is not a concern at prototype scale.

8. Discussion and Analysis

This section reflects on the design decisions made throughout the project, considers the trade-offs involved, and evaluates the extent to which the system meets its original specification.

8.1 Design Decision: Non-Blocking over Interrupts

An early design choice was whether to use hardware interrupts or a polling-based `millis()` approach for input handling. Interrupts offer lower latency but introduce complexity around shared data and re-entrancy. Given the relatively low reaction speed required (human input is naturally limited to ~50 ms), the `millis()` polling approach was judged sufficient and resulted in significantly cleaner, more maintainable code.

8.2 State Machine Benefits

Adopting a finite state machine from the outset proved invaluable. It made the addition of new states (such as sleep mode and the proof step in The Challenger) straightforward, as each state was self-contained. Without this architecture, the code would have quickly become tangled with nested conditionals.

8.3 Challenges Encountered

- **wiring issues**
- **buttons would break**
- **troubleshooting code**
- **I2C LCD would have response delay**
- **needed to weld the wires because they would snap**
- **needed a lot of tape and adhesive glues to keep everything in place**
- **timing bugs**

8.4 Future Improvements

Several enhancements could extend the project in future iterations:

- **Wireless Play:** Replacing wired buttons with wireless modules (e.g., nRF24L01) to allow side-by-side or remote play.
- **Graphical Display:** Upgrading to a small OLED or TFT display would allow richer visual feedback and animations.
- **Wear-Levelling:** Implementing a rotating EEPROM write scheme to extend the lifespan of persistent storage.
- **More Game Modes:** Adding a speed-only mode or a timed survival challenge would improve replay value.
- **PCB Design:** Moving from breadboard to a custom PCB would dramatically improve reliability and presentation.

9. Conclusion

This project successfully delivered a fully functional microcontroller-based gaming console meeting all core and extended requirements of the specification. All four gameplay modes Solo Sprint, Synchronous Duel, and The Challenger were implemented and verified through systematic testing.

The software architecture, built around a finite state machine and non-blocking timing, proved robust and scalable throughout development. Hardware integration was clean, with all eight buttons reliably debounced and all peripherals operating as intended.

The experience reinforced the value of disciplined software structure in embedded development. When a system grows in complexity as this one did with the addition of sleep mode, difficulty scaling, and multi-step game modes a well-defined architecture is what keeps the codebase manageable. The non-blocking approach in particular was a key learning: the instinct to reach for delay() is strong, but overcoming it leads to a fundamentally better system.

10. References

All references should be formatted in IEEE style. Add or remove entries as appropriate for your project.

- [1] Arduino LLC, *Arduino Reference Documentation*, 2024. [Online]. Available: <https://www.arduino.cc/reference/en/>
- [2] Microchip Technology, *ATmega328P Datasheet*, Microchip Technology Inc., 2016.
- [3] M. Barr and A. Massa, *Programming Embedded Systems with C and GNU Development Tools*, 2nd ed. Sebastopol, CA: O'Reilly Media, 2006.
- [4] Hitachi Semiconductor, *HD44780 LCD Controller Datasheet*. [Online]. Available: <https://www.sparkfun.com/datasheets/LCD/HD44780.pdf>

Box Design

